

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

**«ЗАПРОСЫ НА ВЫБОРКУ И МОДИФИКАЦИЮ ДАННЫХ. ПРЕДСТАВЛЕНИЯ. РАБОТА С
ИНДЕКСАМИ»**

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Барецкий Максим Степанович

Факультет прикладной информатики

Группа К3241

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2025/2026
Цель работы

Целью лабораторной работы является овладение практическими навыками создания запросов на выборку данных к базе данных PostgreSQL, создания представлений, использования подзапросов при модификации данных, а также создания и анализа индексов с помощью команды EXPLAIN.

Практическое задание

В ходе выполнения лабораторной работы необходимо:

1. Создать запросы на выборку данных к базе данных PostgreSQL согласно индивидуальному заданию.
2. Создать представления на основе запросов.
3. Составить три запроса на модификацию данных: INSERT, UPDATE, DELETE с использованием подзапросов.
4. Изучить графическое представление запросов и историю запросов в pgAdmin 4.
5. Создать простой и составной индексы для двух запросов.
6. Сравнить время выполнения запросов без индексов и с индексами с помощью EXPLAIN ANALYZE.
7. Удалить созданные индексы.

Схема базы данных

В лабораторной работе используется база данных, разработанная в предыдущей лабораторной работе для предметной области «Сеть автомастерских».

База данных содержит сведения о мастерских, мастерах, клиентах, автомобилях, договорах на ремонт, выполняемых услугах и используемых запчастях.

Основные сущности базы данных:

- workshops — мастерские;
- masters — мастера;
- clients — клиенты;
- cars — автомобили клиентов;
- contracts — договоры на ремонт;
- services — виды услуг;
- contract_services — услуги, включенные в договор;
- parts — запчасти;
- contract_parts — запчасти, использованные в рамках договора.



Рисунок 1 – ERD-схема

Выполнение работы

1. Создание запросов к базе данных

Запросы выполнялись в инструменте Query Tool среды pgAdmin 4. Для каждого запроса была введена SQL-команда, после чего результат выполнения отображался на вкладке Data Output.

Запрос 1.

Выбрать фамилию того механика, который чаще всех работает с автомобилями марки ”Тойота”.

Dashboard X Properties X SQL X Statistics X Dependencies X Dependents X Processes X Untitled* X automasterskaya/bareckijmaksim@Postgres X

automasterskaya/bareckijmaksim@Postgres

No limit

Query Query History AI Assistant

```
1 WITH mechanic_counts AS (  
2     SELECT  
3         e.employee_id,  
4         e.full_name,  
5         COUNT(*) AS cnt  
6     FROM contract_line cl  
7     JOIN contract c ON c.contract_id = cl.contract_id  
8     JOIN car ca ON ca.car_id = c.car_id  
9     JOIN car_model cm ON cm.model_id = ca.model_id  
10    JOIN employee e ON e.employee_id = cl.employee_id  
11    JOIN employee_assignment ea ON ea.employee_id = e.employee_id  
12    JOIN employee_category ec ON ec.category_id = ea.category_id  
13    WHERE cm.brand = 'Toyota'  
14    AND ec.position_name = 'Мастер'  
15    GROUP BY e.employee_id, e.full_name  
16 ),  
17 max_count AS (  
18     SELECT MAX(cnt) AS max_cnt FROM mechanic_counts  
19 )  
20 SELECT mc.full_name  
21 FROM mechanic_counts mc  
22 JOIN max_count m ON mc.cnt = m.max_cnt;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	full_name character varying (100)
1	Петров Алексей Николаев...

Total rows: 1 Query complete 00:00:00.089 LF Ln 22, Col 40

Рисунок 2 – Результат выполнения запроса 1

Запрос 2

Определить тех владельцев автомобилей, которых всегда обслуживает один и тот же мастер (в зависимости от вида работ). Вывести фамилию механика и его постоянных клиентов.

The screenshot shows a SQL query editor with the following query:

```
1 SELECT
2     e.full_name AS mechanic_name,
3     clt.full_name AS client_name
4 FROM contract c
5 JOIN contract_line cl ON cl.contract_id = c.contract_id
6 JOIN employee e ON e.employee_id = cl.employee_id
7 JOIN client clt ON clt.client_id = c.client_id
8 JOIN employee_assignment ea ON ea.employee_id = e.employee_id
9 JOIN employee_category ec ON ec.category_id = ea.category_id
10 WHERE ec.position_name = 'Мастер'
11 GROUP BY clt.client_id, clt.full_name, e.employee_id, e.full_name
12 HAVING COUNT(DISTINCT cl.employee_id) = 1
13 ORDER BY mechanic_name, client_name;
```

Below the query editor, the 'Data Output' tab shows the results of the query. The results are displayed in a table with two columns: 'mechanic_name' and 'client_name'. The table shows three rows of data:

	mechanic_name character varying (100)	client_name character varying (100)
1	Кузнецов Илья Павлович	Егорова Ольга Петровна
2	Петров Алексей Николаев...	Смирнов Андрей Викторов...
3	Сидоров Максим Олегович	Егорова Ольга Петровна

At the bottom of the interface, the status bar indicates: 'Total rows: 3', 'Query complete 00:00:00.050', 'LF', and 'Ln 13, Col 37'.

Рисунок 3 – Результат запроса 2

Запрос 3

Вывести фамилии механиков, которые не выполняли работы в срок и количество дней просрочки выполнения заказа.

QueryQuery HistoryAI Assistant

12345678910111213

```
SELECT
  e.full_name AS mechanic_name,
  c.contract_id,
  (COALESCE(c.actual_finish_date, DATE '2025-04-01') - c.planned_finish_date) AS overdue_days
FROM contract c
JOIN contract_line cl ON cl.contract_id = c.contract_id
JOIN employee e ON e.employee_id = cl.employee_id
JOIN employee_assignment ea ON ea.employee_id = e.employee_id
JOIN employee_category ec ON ec.category_id = ea.category_id
WHERE ec.position_name = 'Мастер'
  AND c.planned_finish_date IS NOT NULL
  AND COALESCE(c.actual_finish_date, DATE '2025-04-01') > c.planned_finish_date
ORDER BY overdue_days DESC, mechanic_name;
```

Data OutputMessagesNotifications

Showing rows: 1 to 2Page No: 1 of 1

	mechanic_name character varying (100)	contract_id integer	overdue_days integer
1	Кузнецов Илья Павлович	2	24
2	Сидоров Максим Олегов...	2	24

Total rows: 2Query complete 00:00:00.050LF Ln 13, Col 43

Рисунок 4 – Результат запроса 3

Запрос 4

Вывести данные клиентов, которые максимально часто посещали автосервис за прошедший год.

QueryQuery HistoryAI Assistant

Execute script

F5

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

WITH yearly_visits AS (

SELECT

clt.client_id,

clt.full_name,

clt.phone,

clt.email,

COUNT(*) AS visit_count

FROM client clt

JOIN contract c ON c.client_id = clt.client_id

WHERE c.acceptance_date >= DATE '2024-04-01'

AND c.acceptance_date < DATE '2025-04-01'

GROUP BY clt.client_id, clt.full_name, clt.phone, clt.email

),

max_visits AS (

SELECT MAX(visit_count) AS max_count

FROM yearly_visits

)

SELECT yv.*

FROM yearly_visits yv

JOIN max_visits mv ON mv.max_count = yv.visit_count;

Data OutputMessagesNotifications

Showing rows: 1 to 2

Page No: 1 of 1

	client_id [PK] integer	full_name character varying (100)	phone character varying (20)	email character varying (100)	visit_count bigint
1	1	Смирнов Андрей Викторов...	+7-999-111-22-33	smirnov@mail.ru	1
2	2	Егорова Ольга Петровна	+7-999-222-33-44	egorova@mail.ru	1

Total rows: 2

Query complete 00:00:00.062

Successfully run. Total query runtime: 62 msec. 2 rows affected.

Рисунок 5 – Результат запроса 4

Запрос 5

Сколько заработал каждый мастер за прошедший месяц?

Open in a new tab

history

AI Assistant

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

```
SELECT
    e.employee_id,
    e.full_name,
    ROUND(
        COALESCE(
            SUM(
                CASE
                    WHEN c.actual_finish_date >= DATE '2025-03-01'
                    AND c.actual_finish_date < DATE '2025-04-01'
                    THEN cl.labor_price * 0.50
                    ELSE 0
                END
            ),
            0
        ),
        2
    ) AS salary_last_month
FROM employee e
JOIN employee_assignment ea ON ea.employee_id = e.employee_id
JOIN employee_category ec ON ec.category_id = ea.category_id
LEFT JOIN contract_line cl ON cl.employee_id = e.employee_id
LEFT JOIN contract c ON c.contract_id = cl.contract_id
WHERE ec.position_name = 'Мастер'
GROUP BY e.employee_id, e.full_name
ORDER BY e.full_name;
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 3

Page No: 1 of 1

Total rows: 3

Query complete 00:00:00.039

LF

Ln 25, Col 22

Рисунок 6 – Результат запроса 5

Запрос 6
Вести данные владельцев автомобилей, которые обращались в ремонт больше одного раза.

QueryQuery HistoryAI Assistant

1234567891011

```
SELECT
  clt.client_id,
  clt.full_name,
  clt.phone,
  clt.email,
  COUNT(c.contract_id) AS repair_count
FROM client clt
JOIN contract c ON c.client_id = clt.client_id
GROUP BY clt.client_id, clt.full_name, clt.phone, clt.email
HAVING COUNT(c.contract_id) > 1
ORDER BY repair_count DESC, clt.full_name;
```

Data OutputMessagesNotifications

Showing rows: 1 to 1Page No: 1 of 1

	client_id [PK] integer	full_name character varying (100)	phone character varying (20)	email character varying (100)	repair_count bigint
1	1	Смирнов Андрей Викторов...	+7-999-111-22-33	smirnov@mail.ru	2

✓ Successfully run. Total query runtime: 112 msec. 1 rows affected. ✕

Total rows: 1Query complete 00:00:00.112LF Ln 11, Col 43

Рисунок 7 – Результат запроса 6

Запрос 7

За каждый день просрочки выполнения заказа механику назначается штраф в размере 5%. Рассчитать штраф каждого механика за прошедший месяц.

QueryQuery HistoryAI Assistant

1SELECT

2e.employee_id,

3e.full_name,

4ROUND(

5COALESCE(

6SUM(

7CASE

8WHEN c.planned_finish_date >= DATE '2025-03-01'

9AND c.planned_finish_date < DATE '2025-04-01'

10AND COALESCE(c.actual_finish_date, DATE '2025-04-01') > c.planned_finish_date

11THEN (cl.labor_price * 0.50) * 0.05 *

12(COALESCE(c.actual_finish_date, DATE '2025-04-01') - c.planned_finish_date)

13ELSE 0

14END

15),

160,

17),

182

19) AS penalty_last_month

20FROM employee e

21JOIN employee_assignment ea ON ea.employee_id = e.employee_id

22JOIN employee_category ec ON ec.category_id = ea.category_id

23LEFT JOIN contract_line cl ON cl.employee_id = e.employee_id

24LEFT JOIN contract c ON c.contract_id = cl.contract_id

25WHERE ec.position_name = 'Мастер'

26GROUP BY e.employee_id, e.full_name

27ORDER BY penalty_last_month DESC, e.full_name;

Data OutputMessagesNotifications

Showing rows: 1 to 3

Page No: 1 of 1

employee_id [PK] integer	full_name character varying (100)	penalty_last_month numeric
1	3 Сидоров Максим Олегович	4620.00
2	4 Кузнецов Илья Павлович	1080.00
3	2 Петров Алексей Николаев...	0.00

✓ Successfully run. Total query runtime: 46 msec. 3 rows affected. ✕

Total rows: 3Query complete 00:00:00.046LF Ln 27, Col 47

Рисунок 8 – Запрос результат 7

2. Создание представлений

Представления создавались с помощью команды CREATE VIEW. После создания каждого представления выполнялся запрос SELECT * FROM view_name, чтобы проверить его содержимое

Представление 1

Для заказчиков (фамилию механика и модель автомобиля, которую он ремонтирует чаще всего);

```
Query Query History AI Assistant
1 CREATE OR REPLACE VIEW vw_mechanic_favorite_car_model AS
2 WITH mechanic_model_stats AS (
3     SELECT
4         e.employee_id,
5         e.full_name AS mechanic_name,
6         cm.brand,
7         cm.model_name,
8         COUNT(*) AS repair_count,
9         ROW_NUMBER() OVER (
10            PARTITION BY e.employee_id
11            ORDER BY COUNT(*) DESC, cm.brand, cm.model_name
12        ) AS rn
13     FROM contract_line cl
14     JOIN contract c ON c.contract_id = cl.contract_id
15     JOIN car ca ON ca.car_id = c.car_id
16     JOIN car_model cm ON cm.model_id = ca.model_id
17     JOIN employee e ON e.employee_id = cl.employee_id
18     JOIN employee_assignment ea ON ea.employee_id = e.employee_id
19     JOIN employee_category ec ON ec.category_id = ea.category_id
20     WHERE ec.position_name = 'Macrep'
21     GROUP BY e.employee_id, e.full_name, cm.brand, cm.model_name
22 )
23 SELECT
24     mechanic_name,
25     brand,
26     model_name,
27     repair_count
28 FROM mechanic_model_stats
29 WHERE rn = 1;
30
```

```
Data Output Messages Notifications
CREATE VIEW

Query returned successfully in 58 msec.
```

Total rows: Query complete 00:00:00.058 LF Ln 30, Col 1



```
1 SELECT * FROM vw_mechanic_favorite_car_model;  
2
```



Showing rows: 1 to 3 Page No: 1 of 1

	mechanic_name character varying (100)	brand character varying (40)	modelName character varying (40)	repair_count bigint
1	Петров Алексей Николаев...	Toyota	Camry	2
2	Сидоров Максим Олегович	Kia	Rio	1
3	Кузнецов Илья Павлович	Kia	Rio	1

Представление 2

Для менеджеров (рассчитать премию все механикам, которые за прошедший месяц все свои заказы выполнили своевременно - 10% от зарплаты).

QueryQuery HistoryAI Assistant

```
1 SET search_path TO autoservice;
2
3 CREATE OR REPLACE VIEW vw_mechanic_monthly_bonus AS
4 WITH monthly_jobs AS (
5     SELECT
6         e.employee_id,
7         e.full_name,
8         cl.labor_price,
9         c.planned_finish_date,
10        c.actual_finish_date
11     FROM employee e
12    JOIN contract_line cl ON cl.employee_id = e.employee_id
13    JOIN contract c ON c.contract_id = cl.contract_id
14    JOIN employee_assignment ea ON ea.employee_id = e.employee_id
15    JOIN employee_category ec ON ec.category_id = ea.category_id
16   WHERE ec.position_name = 'Мастер'
17         AND c.actual_finish_date >= DATE '2025-03-01'
18         AND c.actual_finish_date < DATE '2025-04-01'
19 ),
20 salary_calc AS (
21     SELECT
22         employee_id,
23         full_name,
24         ROUND(SUM(labor_price * 0.50), 2) AS salary_amount,
25         BOOL_AND(actual_finish_date <= planned_finish_date) AS all_in_time
26     FROM monthly_jobs
27    WHERE actual_finish_date IS NOT NULL
28         AND planned_finish_date IS NOT NULL
29    GROUP BY employee_id, full_name
30 )
31 SELECT
32     employee_id,
33     full_name,
34     salary_amount,
35     CASE
36         WHEN all_in_time THEN ROUND(salary_amount * 0.10, 2)
37         ELSE 0.00
38     END AS bonus_amount
39 FROM salary_calc;
```

Data OutputMessagesNotifications

CREATE VIEW

Query returned successfully in 32 msec.

Total rows: Query complete 00:00:00.032LF Ln 39, Col 18

QueryQuery HistorySort/Filter

1 SELECT * FROM vw_mechanic_monthly_bonus;

Data OutputMessagesNotifications

Showing rows: 1 to 1Page No: 1of 1

employee_id	full_name	salary_amount	bonus_amount
integer	character varying (100)	numeric	numeric
1	Петров Алексей Николаев...	1800.00	180.00

Total rows: 1Query complete 00:00:00.047

Successfully run. Total query runtime: 47 msec. 1 rows affected.

LF Ln 1, Col 41

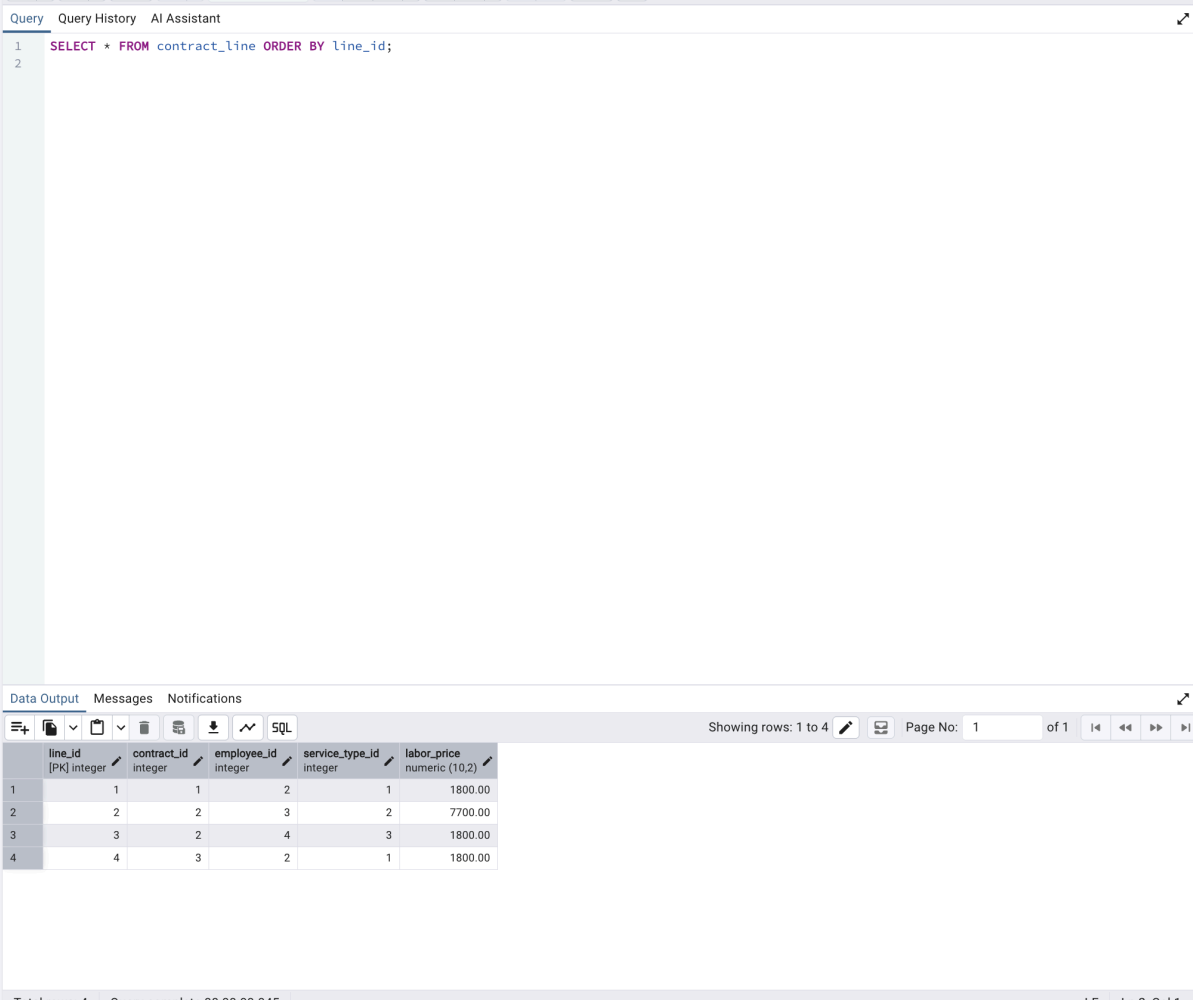
3. Запросы на модификацию данных с использованием подзапросов

Для выполнения данного пункта были составлены запросы INSERT, UPDATE и DELETE, в которых используются подзапросы. Перед и после выполнения каждого запроса выполнялся SELECT, чтобы сравнить состояние данных.

3.1. INSERT с использованием подзапроса

Формулировка запроса:

добавить новый договор для клиента и автомобиля, найденных с помощью подзапросов.



The screenshot shows a database management interface with a query editor and a results pane. The query editor contains the following SQL statement:

```
1 SELECT * FROM contract_line ORDER BY line_id;
2
```

The results pane displays the output of the query as a table with 5 columns: line_id, contract_id, employee_id, service_type_id, and labor_price. The table contains 4 rows of data.

line_id	contract_id	employee_id	service_type_id	labor_price
1	1	1	2	1800.00
2	2	2	3	7700.00
3	3	2	4	1800.00
4	4	3	2	1800.00

The interface also shows a status bar at the bottom indicating "Total rows: 4" and "Query complete 00:00:00.045".

QueryQuery HistoryAI Assistant

1234567891011121314151617181920212223242526

```
INSERT INTO contract_line (contract_id, employee_id, service_type_id, labor_price)
SELECT
  2,
  (
    SELECT e.employee_id
    FROM employee e
    JOIN employee_assignment ea ON ea.employee_id = e.employee_id
    JOIN employee_category ec ON ec.category_id = ea.category_id
    WHERE ec.position_name = 'Мастер'
    AND ec.specialization = 'Диагностика'
    LIMIT 1
  ),
  (
    SELECT service_type_id
    FROM service_type
    WHERE service_name = 'Компьютерная диагностика'
  ),
  (
    SELECT hour_price * norm_hours
    FROM service_price sp
    JOIN service_type st ON st.service_type_id = sp.service_type_id
    WHERE st.service_name = 'Компьютерная диагностика'
    LIMIT 1
  );
```

Data OutputMessagesNotifications

INSERT 0 1

Query returned successfully in 42 msec.

Total rows:Query complete 00:00:00.042

LF Ln 26, Col 1

✓ Query returned successfully in 42 msec. ✕

QueryQuery HistoryAI Assistant

1

SELECT * FROM contract_line ORDER BY line_id;

Data OutputMessagesNotifications

Showing rows: 1 to 5

Page No: 1 of 1

	line_id [PK] integer	contract_id integer	employee_id integer	service_type_id integer	labor_price numeric (10,2)
1	1	1	2	1	1800.00
2	2	2	3	2	7700.00
3	3	2	4	3	1800.00
4	4	3	2	1	1800.00
5	5	2	4	3	1800.00

✓ Successfully run. Total query runtime: 55 msec. 5 rows affected.

Total rows: 5Query complete 00:00:00.055LF Ln 1, Col 46

3.2. UPDATE с использованием подзапроса

Формулировка запроса:

увеличить стоимость работ на 10% для услуг, которые относятся к конкретному виду ремонта.

QueryQuery HistoryAI Assistant

```
1 SELECT contract_id, actual_finish_date, contract_status
2 FROM contract
3 ORDER BY contract_id;
```

Data OutputMessagesNotifications

Showing rows: 1 to 3Page No: 1 of 1

	contract_id [PK] integer	actual_finish_date date	contract_status character varying (20)
1	1	2025-03-03	завершён
2	2	2025-03-08	в работе
3	3	2025-03-12	завершён

Total rows: 3Query complete 00:00:00.055LF Ln 3, Col 22

QueryQuery HistoryAI Assistant

Execute script
F5

```
1  UPDATE contract
2  SET contract_status = 'завершён'
3  WHERE contract_id IN (
4      SELECT contract_id
5      FROM contract
6      WHERE actual_finish_date IS NOT NULL
7            AND contract_status <> 'завершён'
8  );
```

Data OutputMessagesNotifications

UPDATE 1

Query returned successfully in 46 msec.

Total rows: Query complete 00:00:00.046

Ln 8, Col 1

✓ Query returned successfully in 46 msec. ✕

```
1 SELECT contract_id, actual_finish_date, contract_status
2 FROM contract
3 ORDER BY contract_id;
```

	contract_id [PK] integer	actual_finish_date date	contract_status character varying (20)
1	1	2025-03-03	завершён
2	2	2025-03-08	завершён
3	3	2025-03-12	завершён

✓ Successfully run. Total query runtime: 57 msec. 3 rows affected. ✕

3.3. DELETE с использованием подзапроса

Формулировка запроса:

удалить из таблицы использованных запчастей записи, относящиеся к договорам, которые не содержат выполненных услуг.

QueryQuery HistoryAI Assistant

12

SELECT * FROM used_part ORDER BY used_part_id;

Data OutputMessagesNotifications

Showing rows: 1 to 3 of 1

	used_part_id [PK] integer	line_id integer	part_id integer	quantity integer	supply_source character varying (20)	price_at_repair numeric (10,2)
1	1	1	1	1	автомастерская	650.00
2	2	1	2	1	автомастерская	3200.00
3	3	2	3	2	клиент	0.00

Total rows: 3

Query complete 00:00:00.058

LF

Ln 2, Col 1

QueryQuery HistoryAI Assistant

Execute scriptFS

```
1 DELETE FROM used_part
2 WHERE line_id IN (
3     SELECT cl.line_id
4     FROM contract_line cl
5     JOIN contract c ON c.contract_id = cl.contract_id
6     WHERE c.contract_status = 'отменён'
7 )
8 AND supply_source = 'клиент';
```

Data OutputMessagesNotifications

DELETE 1

Query returned successfully in 59 msec.

Total rows:Query complete 00:00:00.059

✓ Query returned successfully in 59 msec. ✕

LF Ln 8, Col 30

QueryQuery HistoryAI Assistant

1SELECT * FROM used_part ORDER BY us

Execute script

Data OutputMessagesNotifications

Showing rows: 1 to 2Page No: 1 of 1

	used_part_id [PK] integer	line_id integer	part_id integer	quantity integer	supply_source character varying (20)	price_at_repair numeric (10,2)
1	1	1	1	1	автомастерская	650.00
2	2	1	2	1	автомастерская	3200.00

LF Ln 1, Col 47

Total rows: 2Query complete 00:00:00.039

✓ Successfully run. Total query runtime: 39 msec. 2 rows affected. ✕

4. Изучение графического представления запросов и истории запросов

В pgAdmin 4 был использован инструмент Query Tool. На верхней панели инструмента вводились SQL-команды. После выполнения запросов результаты отображались на вкладке Data Output, а сообщения сервера — на вкладке Messages.

Для просмотра плана выполнения запроса использовалась вкладка Explain. С помощью команды EXPLAIN и режима EXPLAIN ANALYZE был получен план выполнения запроса. В графическом представлении отображались операции, которые PostgreSQL использует для выполнения запроса, например Seq Scan, Index Scan, Hash Join, Nested Loop.

Также была просмотрена вкладка Query History, где отображаются ранее выполненные SQL-команды, время выполнения, количество возвращенных строк и сообщения сервера.

5. Работа с индексами

Для анализа индексов были выбраны два запроса. Сначала запросы выполнялись без дополнительных индексов, затем создавались индексы, после чего запросы выполнялись повторно. Для анализа использовалась команда EXPLAIN ANALYZE.

5.1. Простой индекс

Формулировка запроса:

найти договоры по конкретной дате заключения.

Рисунок – Без индексов

automasterskaya/bareckijmaksim@Postgres

Query

Query History

AI Assistant

1

EXPLAIN ANALYZE

2

SELECT

3

c.contract_id,

4

c.acceptance_date,

5

c.contract_status,

6

clt.full_name

7

FROM contract c

8

JOIN client clt ON clt.client_id = c.client_id

9

WHERE c.contract_status = 'a pafore'

10

AND c.acceptance_date >= DATE '2025-01-01'

11

ORDER BY c.acceptance_date;

Data Output

Messages

Notifications

Showing rows: 1 to 11

Page No: 1

of 1

QUERY PLAN

text

1

Sort (cost=25.03..25.03 rows=1 width=284) (actual time=0.176..0.177 rows=0 loops=1)

2

Sort Key: c.acceptance_date

3

Sort Method: quicksort Memory: 25kB

4

-> Nested Loop (cost=0.14..25.02 rows=1 width=284) (actual time=0.156..0.156 rows=0 loops=1)

5

-> Seq Scan on contract c (cost=0.00..16.75 rows=1 width=70) (actual time=0.155..0.155 rows=0 loops=1)

6

Filter: ((acceptance_date >= '2025-01-01'::date) AND ((contract_status)::text = 'a pafore'::text))

7

Rows Removed by Filter: 3

8

-> Index Scan using client_pkey on client clt (cost=0.14..8.16 rows=1 width=222) (never executed)

9

Index Cond: (client_id = c.client_id)

10

Planning Time: 2.378 ms

11

Execution Time: 0.218 ms

Total rows: 11

Query complete 00:00:00.035

LF

Ln 11, Col 28

Рисунок – Прописываю индексы

[illegible]

Рисунок – С индексами

Dashboard X Properties X SQL X Statistics X Dependencies X Dependents X Processes X Untitled* X automasterskaya/bareckijmaksim@Postgres* X

automasterskaya/bareckijmaksim@Postgres

Query Query History AI Assistant

```
1 EXPLAIN ANALYZE
2 SELECT
3     c.contract_id,
4     c.acceptance_date,
5     c.contract_status,
6     clt.full_name
7 FROM contract c
8 JOIN client clt ON clt.client_id = c.client_id
9 WHERE c.contract_status = 'a pafore'
10 AND c.acceptance_date >= DATE '2025-01-01'
11 ORDER BY c.acceptance_date;
```

Data Output Messages Notifications

Showing rows: 1 to 11

Page No: 1 of 1

QUERY PLAN

text

```
1 Sort (cost=9.32..9.33 rows=1 width=284) (actual time=0.044..0.045 rows=0 loops=1)
2   Sort Key: c.acceptance_date
3   Sort Method: quicksort Memory: 25kB
4   -> Nested Loop (cost=0.14..9.31 rows=1 width=284) (actual time=0.016..0.017 rows=0 loops=1)
5     -> Seq Scan on contract c (cost=0.00..1.04 rows=1 width=70) (actual time=0.015..0.016 row=0)
6       Filter: ((acceptance_date >= '2025-01-01'::date) AND ((contract_status)::text = 'a pafore'::text))
7       Rows Removed by Filter: 3
8     -> Index Scan using client_pkey on client clt (cost=0.14..8.16 rows=1 width=222) (never executed)
9       Index Cond: (client_id = c.client_id)
10 Planning Time: 0.514 ms
11 Execution Time: 0.082 ms
```

Total rows: 11 Query complete 00:00:00.040

LF Ln 11, Col 28

5.2. Составной индекс

Формулировка запроса:

найти договоры по мастерской и дате заключения.

Рисунок — Без индексов

The screenshot displays a PostgreSQL query editor interface. The top navigation bar includes tabs for Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, Processes, and an active tab for 'automasterskaya/bareckijmaksim@Postgres'. Below the navigation bar is a toolbar with icons for saving, running, and other query-related actions. The main query editor area contains the following SQL query:

```
1 EXPLAIN ANALYZE
2 SELECT
3   cl.contract_id,
4   st.service_name,
5   SUM(cl.labor_price)
6 FROM contract_line cl
7 JOIN service_type st ON st.service_type_id = cl.service_type_id
8 GROUP BY cl.contract_id, st.service_name
9 ORDER BY cl.contract_id;
```

Below the query editor is the 'Data Output' section, which shows the 'QUERY PLAN' for the executed query. The plan details the execution steps, including a GroupAggregate, a Sort operation, a Hash Join, and a Seq Scan on the contract_line table. The plan also includes cost estimates, row counts, and execution times for each step.

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Loops
1	GroupAggregate	(cost=111.18..141.78 rows=1360 width=154)	1360	154	1.344..1.347	4	1
2	Group Key: cl.contract_id, st.service_name						
3	-> Sort	(cost=111.18..114.58 rows=1360 width=138)	1360	138	1.295..1.296	5	1
4	Sort Key: cl.contract_id, st.service_name						
5	Sort Method: quicksort Memory: 25kB						
6	-> Hash Join	(cost=13.15..40.40 rows=1360 width=138)	1360	138	1.212..1.216	5	1
7	Hash Cond: (cl.service_type_id = st.service_type_id)						
8	-> Seq Scan on contract_line cl	(cost=0.00..23.60 rows=1360 width=24)	1360	24	0.241..0.242	5	1
9	-> Hash	(cost=11.40..11.40 rows=140 width=122)	140	122	0.943..0.944	3	1
10	Buckets: 1024 Batches: 1 Memory Usage: 9kB						
11	-> Seq Scan on service_type st	(cost=0.00..11.40 rows=140 width=122)	140	122	0.916..0.917	3	1
12	Planning Time	2.209 ms					
13	Execution Time	1.390 ms					

At the bottom of the interface, there is a status bar showing 'Total rows: 13' and 'Query completed: 00:00:00.043'.

Рисунок – Прописываю

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × Untitled* × automasterskaya/bareckijmaksim@Postgres*

automasterskaya/bareckijmaksim@Postgres

📁

🔍

✎

⌵

No limit

⏮

⏪

⏩

⏭

🖨

🗑

⚙

☰

💡

Query Query History AI Assistant

1 CREATE INDEX idx_contract_line_contract_service

2 ON contract_line (contract_id, service_type_id);

3

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 47 msec.

Total rows: Query complete 00:00:00.047

✓ Query returned successfully in 47 msec. ✕ LF Ln 3, Col 4

Рисунок – С индексами

The screenshot displays a PostgreSQL query editor interface. The top bar shows the database name 'automasterskaya/bareckijmaksim@Postgres'. The query editor contains the following SQL code:

```
1 EXPLAIN ANALYZE
2 SELECT
3     cl.contract_id,
4     st.service_name,
5     SUM(cl.labor_price)
6 FROM contract_line cl
7 JOIN service_type st ON st.service_type_id = cl.service_type_id
8 GROUP BY cl.contract_id, st.service_name
9 ORDER BY cl.contract_id;
```

Below the query editor, the 'Data Output' tab is active, showing the 'QUERY PLAN' for the executed query. The plan details are as follows:

Step	Operation	Cost	Time	Rows	Width	Loops
1	GroupAggregate	(cost=13.15..13.26)	(actual time=0.085..0.089)	rows=5	width=154	loops=1
2	Group Key	cl.contract_id, st.service_name				
3	Sort	(cost=13.15..13.16)	(actual time=0.074..0.076)	rows=5	width=138	loops=1
4	Sort Key	cl.contract_id, st.service_name				
5	Sort Method: quicksort	Memory: 25kB				
6	Hash Join	(cost=1.11..13.09)	(actual time=0.042..0.045)	rows=5	width=138	loops=1
7	Hash Cond	(st.service_type_id = cl.service_type_id)				
8	Seq Scan on service_type st	(cost=0.00..11.40)	(actual time=0.006..0.007)	rows=3	width=122	loops=1
9	Hash	(cost=1.05..1.05)	(actual time=0.028..0.029)	rows=5	width=24	loops=1
10	Buckets	1024 Batches: 1	Memory Usage: 9kB			
11	Seq Scan on contract_line cl	(cost=0.00..1.05)	(actual time=0.007..0.008)	rows=5	width=24	loops=1
12	Planning Time	0.482 ms				
13	Execution Time	0.131 ms				

The bottom status bar indicates 'Total rows: 13' and 'Query complete 00:00:00.0045'.

6. Выводы

В ходе выполнения лабораторной работы были получены практические навыки работы с запросами на выборку данных в PostgreSQL. Были составлены SQL-запросы с использованием соединений таблиц, группировки, агрегатных функций и общих табличных выражений.

Также были созданы представления, которые позволяют упростить дальнейшую работу с часто используемыми запросами. Представления были проверены с помощью команды SELECT.

Были выполнены запросы на модификацию данных INSERT, UPDATE и DELETE с использованием подзапросов. Перед и после выполнения

запросов выполнялась проверка данных, что позволило оценить изменения в таблицах.

Кроме того, были изучены планы выполнения запросов с помощью команды EXPLAIN ANALYZE и графического представления во вкладке Explain в pgAdmin 4. Для двух запросов были созданы простой и составной индексы. После сравнения времени выполнения запросов было установлено, что индексы могут ускорять поиск данных по часто используемым условиям отбора. После завершения анализа созданные индексы были удалены.